الاحتراف الكامل

في مكتبات بايثون لعلم البيانات والتعلّم الآلي

الجزء الرابع: مكتبة Pandas

الفصول ٧ - ١١: الأمثلة العملية، الأداء، التصحيح، أفضل الممارسات، والمشاريع المصغّرة

مرجع احترافي للمبرمجين والمهندسين وعلماء البيانات

جدول المحتويات

جدول المحتويات 2
الفصل السابع: الأمثلة العملية 4
٧.١ المستوى المبتدئ 4
مثال ١: تنظيف جدول موظفين بسيط 4
٧.٢ المستوى المتوسط 4
مثال ٢: تحليل مبيعات شهرية حسب المنتج 4
٧.٣ المستوى الاحترافي 4
مثال ٣: بناء خط أنابيب تنظيف كامل بأسلوب Method Chaining 4
٧.٤ مثال من مشروع حقيقي: تقرير أداء مبيعات 5
تمارين عملية 5
حلول مقترحة (تلميحات) 6
الفصل الثامن: الأداء 7
٨.١ لماذا تُصبح Pandas بطيئة أحيانًا 7
٨.٢ vectorized مقابل apply: قياس فعلي 7
٨.٣ اختيار dtype الصحيح 7
٨.٤ قراءة الملفات بكفاءة 7
٨.٥ أدوات القياس 8
أفضل الممارسات 8
الأخطاء الشائعة 8
الفصل التاسع: التصحيح 9
٩.١ SettingWithCopyWarning بالتفصيل 9
٩.٢ رسائل الخطأ الشائعة 9
٩.٣ استراتيجية تشخيص خطوة بخطوة 9
الأخطاء الشائعة 10
الأسئلة الشائعة 10
الفصل العاشر: أفضل الممارسات 11
١٠.١ أسلوب كتابة احترافي 11
١٠.٢ هيكلة خط أنابيب البيانات (Pipeline) 11
١٠.٣ الاختبار 11
١٠.٤ التوثيق وقابلية الصيانة 11
أفضل الممارسات 12
الفصل الحادي عشر: مشاريع مصغّرة 13
مشروع ١: تحليل مبيعات متجر إلكتروني 13

13 ..... مشروع ٢: دمج بيانات عملاء من مصدرين وكشف التكرار
13 ..... مشروع ٣: لوحة مؤشرات أداء بسيطة (KPI Summary)
14 ..... تمارين عملية
14 ..... حلول مقترحة (تلميحات)
14 ..... مراجع وروابط رسمية
14 ..... خاتمة Pandas

الفصل السابع: الأمثلة العملية

٧.١ المستوى المبتدئ

مثال ١: تنظيف جدول موظفين بسيط

```
import pandas as pd

df = pd.DataFrame({
    'name': ['Ali', 'Sara', None, 'Omar'],
    'salary': [5000, None, 6200, 5800],
})

df['name'] = df['name'].fillna('Unknown')
df['salary'] = df['salary'].fillna(df['salary'].mean())
print(df)
```

شرح الكود: fillna تعبئ القيم المفقودة، وهنا نستخدم استراتيجيتين مختلفتين: قيمة ثابتة للنصوص، والمتوسط الحسابي للأرقام — وهو نمط شائع جدًا في مرحلة تنظيف البيانات الأولى.

٧.٢ المستوى المتوسط

مثال ٢: تحليل مبيعات شهرية حسب المنتج

```
sales = pd.DataFrame({
    'date': pd.date_range('2024-01-01', periods=90, freq='D'),
    'product': ['A', 'B', 'C'] * 30,
    'amount': (pd.Series(range(90)) % 50) + 10,
})

monthly = (
    sales
    .set_index('date')
    .groupby('product')
    .resample('ME')['amount']
    .sum()
    .reset_index()
)
print(monthly)
```

شرح الكود: هذا مثال على GroupBy مزدوج المستوى — تجميع حسب المنتج أولاً، ثم إعادة تجميع زمني شهري داخل كل منتج عبر resample، وهو نمط شائع جدًا في تقارير الأعمال.

٧.٣ المستوى الاحترافي

مثال ٣: بناء خط أنابيب تنظيف كامل بأسلوب Method Chaining

```
raw = pd.DataFrame({
    'customer': [' Ali ', 'SARA', 'omar', 'Ali'],
    'amount': [100, -5, 250, 100],
    'date': ['2024-01-05', '2024-13-40', '2024-02-10', '2024-01-05'],
```

```
})

clean = (
    raw
    .assign(customer=lambda d: d['customer'].str.strip().str.title())
    .assign(date=lambda d: pd.to_datetime(d['date'], errors='coerce'))
    .query('amount > 0')
    .dropna(subset=['date'])
    .drop_duplicates()
    .reset_index(drop=True)
)
print(clean)
```

شرح الكود: assign يضيف/يعدّل أعمدة داخل السلسلة دون كسرها، و'errors='coerce في to_datetime يحوّل أي تاريخ غير صالح إلى NaT بدل رفع استثناء يوقف الكود بأكمله، ثم dropna تتخلص من تلك الصفوف الفاسدة لاحقًا. هذا مثال نموذجي على خط أنابيب تنظيف احترافي بأسلوب تعبيري واحد قابل للقراءة من أعلى لأسفل.

٧.٤ مثال من مشروع حقيقي: تقرير أداء مبيعات

```
orders = pd.DataFrame({
    'order_id': range(1, 11),
    'region':
['North','South','North','East','South','North','East','South','North','East'],
    'amount': [250,400,150,300,600,220,180,450,320,270],
})

report = (
    orders
    .groupby('region')
    .agg(total_sales=('amount', 'sum'), avg_order=('amount', 'mean'),
orders_count=('order_id', 'count'))
    .sort_values('total_sales', ascending=False)
    .assign(pct_of_total=lambda d: (d['total_sales'] / d['total_sales'].sum() *
100).round(1))
)
print(report)
```

شرح الكود: agg بأسماء مخصصة تُنتج تقريرًا واضح الأعمدة مباشرة، وassign الأخير يحسب النسبة المئوية من الإجمالي لكل منطقة اعتمادًا على عمود حُسب للتو في نفس السلسلة — توضيح عملي لقوة Method Chaining في بناء تقارير كاملة بلا متغيرات وسيطة.

تمارين عملية

١. لديك جدول طلبات به أعمدة (تاريخ، عميل، مبلغ). احسب أعلى ٣ عملاء إنفاقًا إجماليًا باستخدام groupby وsort_values.

٢. نظّف عمود بريد إلكتروني يحتوي مسافات وأحرف كبيرة وصغيرة مختلطة باستخدام محور str.

٣. ابنِ تقرير مبيعات ربع سنوي (Quarterly) من بيانات يومية باستخدام resample('QE').

حلول مقترحة (تلميحات)

١. df.groupby('عميل')['مبلغ'].sum().sort_values(ascending=False).head(3)

٢. df['email'].str.strip().str.lower()

٣. df.set_index('date')['amount'].resample('QE').sum()

الفصل الثامن: الأداء

٨.١ لماذا تُصبح Pandas بطيئة أحيانًا

أشهر سبب لبطء Pandas في الممارسة العملية هو استخدام apply مع دالة بايثون يدوية بدل العمليات المدمجة المتجهة. سبب آخر شائع هو التوسيع التدريجي لـ DataFrame داخل حلقة (مثل df = df.append(...)) في حلقة)، والذي يُعيد تخصيص الذاكرة بالكامل في كل تكرار.

٨.٢ vectorized مقابل apply: قياس فعلي

```
import time
import numpy as np

n = 500_000
df = pd.DataFrame({'x': np.random.rand(n)})

start = time.perf_counter()
df['y1'] = df['x'].apply(lambda v: v * 2 + 1)   # apply – بطيء نسبيًا
print('apply:', time.perf_counter() - start)

start = time.perf_counter()
df['y2'] = df['x'] * 2 + 1                       # متجهة – سريعة جدًا
print('vectorized:', time.perf_counter() - start)
```

٨.٣ اختيار dtype الصحيح

الحالة	التوصية
نصوص متكررة (فئات محدودة)	astype('category')
أعداد صحيحة كبيرة النطاق غير مطلوبة	downcast إلى int32 أو حتى int8/int16 عبر pd.to_numeric(..., downcast='integer')
أعمدة float بدقة غير حرجة	float32 بدل float64 الافتراضي

```
df['small_int'] = pd.to_numeric(df['small_int'], downcast='integer')
print(df.memory_usage(deep=True))
```

٨.٤ قراءة الملفات بكفاءة

```
# تحديد الأعمدة المطلوبة فقط بدل الملف كاملاً
df = pd.read_csv('big.csv', usecols=['id', 'amount', 'date'])

# تحديد dtype مسبقًا يمنع Pandas من استنتاجه (أبطأ ويقلل الذاكرة)
df = pd.read_csv('big.csv', dtype={'id': 'int32', 'amount': 'float32'})

# قراءة ملف ضخم على دفعات (Chunks)
for chunk in pd.read_csv('huge.csv', chunksize=100_000):
    process(chunk)
```

٨.٥ أدوات القياس

```
%timeit df['x'] * 2
df.memory_usage(deep=True)    # deep=True ضروري لقياس الأعمدة النصية بدقة
```

أفضل الممارسات

- فضّل العمليات المتجهة المدمجة (str, dt, الحساب المباشر) على apply يدوي.
- حدّد usecols وdtype عند قراءة ملفات كبيرة بدل ترك الاستنتاج التلقائي.
- استخدم chunksize للملفات الأكبر من الذاكرة المتاحة.
- حوّل الأعمدة الفئوية إلى category لتوفير ذاكرة كبيرة.

الأخطاء الشائعة

- استدعاء df.append() داخل حلقة (وهي أصلاً مُهملة/محذوفة في الإصدارات الحديثة) بدل تجميع القوائم ثم concat.pd مرة واحدة في النهاية.
- عدم تمرير deep=True عند قياس df.memory_usage() للأعمدة النصية، مما يُظهر رقمًا مضلِّلاً أقل من الحقيقة.

الفصل التاسع: التصحيح

٩.١ SettingWithCopyWarning بالتفصيل

هذا التحذير هو الأكثر إرباكًا للمبتدئين في Pandas. يظهر عندما تحاول Pandas تعديل ما قد يكون View أو Copy من DataFrame أصلي دون تأكيد واضح لنيّتك، وذلك بسبب غموض ناتج عن الفهرسة المتسلسلة (Chained Indexing).

```
# يُصدر تحذيرًا – النية غامضة
df[df['age'] > 25]['status'] = 'adult'

# الحل الصحيح: loc في خطوة واحدة
df.loc[df['age'] > 25, 'status'] = 'adult'

# أو نسخة صريحة إن كنت تعمل على جدول فرعي منفصل عمدًا
subset = df[df['age'] > 25].copy()
subset['status'] = 'adult'
```

٩.٢ رسائل الخطأ الشائعة

رسالة الخطأ	السبب الشائع
'KeyError: 'col_name	اسم عمود غير موجود، غالبًا بسبب مسافات زائدة أو حالة أحرف مختلفة
ValueError: cannot merge... on key with different dtypes	أعمدة الدمج بأنواع مختلفة (مثل int في جدول وstr في آخر)
MergeError: Merge keys are not unique	استخدام validate في merge للتأكد من صحة العلاقة (one-to-one مثلاً)
TypeError عند str.something.	محاولة استخدام محور str على عمود ليس من نوع نصي بالفعل

```
print(df.columns.tolist())      # للتأكد من الأسماء الفعلية للأعمدة
print(df.dtypes)                # للتأكد من الأنواع قبل الدمج

pd.merge(a, b, on='id', validate='one_to_one')  # يرفع خطأ فورًا إن انتهكت العلاقة الافتراض
```

٩.٣ استراتيجية تشخيص خطوة بخطوة

عند فشل سلسلة Method Chaining طويلة، أفضل استراتيجية هي تنفيذها خطوة بخطوة في Jupyter (كل method.() في سطر منفصل مع طباعة .shape وdf.head() بعد كل خطوة) لتحديد الخطوة المسبّبة للمشكلة بدقة، ثم دمجها في سلسلة واحدة بعد التأكد.

```
step1 = raw.dropna(subset=['date'])
print('after dropna:', step1.shape)

step2 = step1.query('amount > 0')
print('after filter:', step2.shape)
```

الأخطاء الشائعة

- تجاهل SettingWithCopyWarning بدل التحول لـ .loc أو .copy() الصريحة.
- الدمج (merge) دون التحقق من dtype الأعمدة المشتركة أولاً.
- افتراض تطابق أسماء الأعمدة دون طباعتها فعليًا (فروق بسيطة كمسافة زائدة تسبب KeyError).

الأسئلة الشائعة

س: كيف أعرف إن كانت نتيجة merge صحيحة الحجم؟

ج: قارن df.shape قبل وبعد الدمج، واستخدم معامل validate لفرض نوع العلاقة المتوقعة (one_to_one, one_to_many...) بحيث يفشل الكود فورًا إن انتهكت البيانات هذا الافتراض.

الفصل العاشر: أفضل الممارسات

١٠.١ أسلوب كتابة احترافي

- استخدم أسماء أعمدة واضحة بصيغة snake_case متسقة (customer_id لا CustomerID أو Customer Id).
- فضّل Method Chaining مع أقواس متعددة الأسطر للتعبيرات الطويلة بدل متغيرات وسيطة كثيرة غير ضرورية.
- استخدم .copy() صراحة عند إنشاء جدول فرعي تنوي تعديله بشكل مستقل.
- استخدم query() للشروط المعقدة القابلة للقراءة بدل فهرسة منطقية متداخلة.

```
# أقل وضوحًا
df[(df['age'] > 25) & (df['city'] == 'Cairo') & (df['salary'] < 10000)]

# أوضح عبر query
df.query("age > 25 and city == 'Cairo' and salary < 10000")
```

١٠.٢ هيكلة خط أنابيب البيانات (Pipeline)

دالة .pipe() تسمح بإدراج دوال تنظيف مخصصة داخل سلسلة Method Chaining بدل كسرها، مما يحافظ على الأسلوب التعبيري حتى مع منطق معقّد يستحق أن يكون دالة منفصلة قابلة لإعادة الاستخدام والاختبار.

```
def remove_outliers(df, column, n_std=3):
    mean, std = df[column].mean(), df[column].std()
    return df[(df[column] - mean).abs() <= n_std * std]

clean = (
    raw
    .pipe(remove_outliers, column='amount')
    .dropna()
    .reset_index(drop=True)
)
```

١٠.٣ الاختبار

```
import pandas.testing as pdt

def test_clean_customer_names():
    raw = pd.DataFrame({'name': [' ali ', 'SARA']})
    result = raw.assign(name=lambda d: d['name'].str.strip().str.title())
    expected = pd.DataFrame({'name': ['Ali', 'Sara']})
    pdt.assert_frame_equal(result, expected)
```

pdt.assert_frame_equal هي الأداة القياسية لمقارنة جداول Pandas في الاختبارات، وتتعامل بذكاء مع فروق التقريب في الأعمدة العشرية ومع ترتيب الفهرس.

١٠.٤ التوثيق وقابلية الصيانة

- وثّق كل دالة تنظيف بيانات بوضوح: ماذا تتوقع كمدخل، وماذا تضمن كمخرج.

- افصل دوال التحويل القابلة لإعادة الاستخدام في وحدة (Module) منفصلة عن السكربت الرئيسي.
- تجنّب السحر الضمني (Implicit Magic): لا تعتمد على ترتيب أعمدة غير موثّق أو فهرس بلا تسمية واضحة.

أفضل الممارسات

- استخدم query() و.pipe() للحفاظ على قابلية القراءة في السلاسل الطويلة.
- اختبر دوال التنظيف بـ pdt.assert_frame_equal.
- وثّق افتراضات كل دالة تحويل بوضوح (الأعمدة المطلوبة، الأنواع المتوقعة).

الفصل الحادي عشر: مشاريع مصغّرة

مشروع ١: تحليل مبيعات متجر إلكتروني

```
import pandas as pd, numpy as np

rng = np.random.default_rng(1)
orders = pd.DataFrame({
    'order_date': pd.date_range('2024-01-01', periods=500, freq='6h'),
    'category': rng.choice(['Electronics','Clothing','Books'], 500),
    'amount': rng.uniform(10, 500, 500).round(2),
})

summary = (
    orders
    .assign(month=lambda d: d['order_date'].dt.to_period('M'))
    .groupby(['month', 'category'])['amount']
    .sum()
    .unstack('category')
    .fillna(0)
)
print(summary)
```

يطبّق هذا المشروع: توليد بيانات زمنية واقعية، محور dt لاستخراج الفترة الشهرية، groupby متعدد المفاتيح، وunstack لتحويل النتيجة من صيغة طويلة إلى جدول عريض جاهز للعرض أو الرسم البياني.

مشروع ٢: دمج بيانات عملاء من مصدرين وكشف التكرار

```
crm = pd.DataFrame({'email': ['ali@x.com','sara@x.com','omar@x.com'], 'name':
['Ali','Sara','Omar']})
newsletter = pd.DataFrame({'email': ['ALI@X.com','new@x.com'], 'subscribed': [True,
True]})

crm['email_norm'] = crm['email'].str.lower()
newsletter['email_norm'] = newsletter['email'].str.lower()

merged = pd.merge(crm, newsletter, on='email_norm', how='outer', indicator=True)
print(merged[['email_norm', 'name', 'subscribed', '_merge']])
```

يطبّق هذا المشروع: تطبيع البيانات قبل الدمج (توحيد حالة الأحرف)، ودمج outer مع indicator=True الذي يضيف عمودًا يوضّح مصدر كل صف (left_only, right_only, both)، وهي أداة تشخيصية ممتازة لفهم نتائج الدمج.

مشروع ٣: لوحة مؤشرات أداء بسيطة (KPI Summary)

```
def kpi_report(df: pd.DataFrame) -> pd.DataFrame:
    return pd.DataFrame({
        'total_revenue': [df['amount'].sum()],
        'avg_order_value': [df['amount'].mean()],
        'orders_count': [len(df)],
        'top_category': [df.groupby('category')['amount'].sum().idxmax()],
    })
```

```
print(kpi_report(orders))
```

يطبّق هذا المشروع: تجميع منطق تقرير كامل داخل دالة واحدة قابلة لإعادة الاستخدام والاختبار، وidxmax لإيجاد تسمية الفئة صاحبة أعلى قيمة بدل القيمة نفسها فقط.

تمارين عملية

١. وسّع مشروع تحليل المبيعات ليحسب أيضًا نسبة نمو كل فئة شهريًا مقارنة بالشهر السابق باستخدام pct_change.

٢. أضف لمشروع الدمج خطوة لعرض السجلات الموجودة في crm فقط دون newsletter عبر تصفية عمود merge_.

٣. طوّر kpi_report لتقبل معامل فترة زمنية وتحسب المؤشرات لكل فترة عبر resample.

حلول مقترحة (تلميحات)

١. summary.pct_change(axis=0) بعد الفهرسة الشهرية.

٢. merged[merged['_merge'] == 'left_only']

٣. استخدم df.set_index('order_date').resample('W').apply(kpi_report_logic).

مراجع وروابط رسمية

- التوثيق الرسمي: https://pandas.pydata.org/docs/
- دليل الأداء: https://pandas.pydata.org/docs/user_guide/enhancingperf.html
- الكود المصدري: https://github.com/pandas-dev/pandas

خاتمة Pandas

بهذا نُتم مرجعًا احترافيًا كاملاً لمكتبة Pandas عبر أحد عشر فصلاً. المكتبة التالية هي Matplotlib، حيث سننتقل من معالجة البيانات إلى تصويرها بصريًا، وهي الخطوة الطبيعية التالية في أي خط أنابيب تحليل بيانات احترافي.